

CSE107 — Lab #2

Basic Arduino Ops

Lab Date: 17/10/2025 — Due: next lab day

Student: Name Surname:	Zeynep Özge Toptaş
Student ID:	240104004006
Section:	2
Instructor:	

1. Objectives

I created an Arduino program that counts button presses while handling issues like button holding and debounce. I also implemented reset functionality as part of the lab exercise.

2. Environment (Arduino IDE)

- **Operating System:** Ubuntu
- **Arduino IDE**
- **Board:** Arduino Uno
- **Language:** Arduino C/C++

3. Command Explanations

• **Serial.begin()** —

Sets up serial communication between the Arduino and the computer. I used it to start communication at a baud rate of 9600.

• **Serial.println()** —

Sends text or variable values to the Serial Monitor with a new line. I used it to display the counter values on the screen.

• **Serial.readString()** —

Reads incoming serial data as a string. I used it to receive reset commands from the Serial Monitor.

• **millis()** —

Returns the number of milliseconds since the program started. I used it to track the last button press and implement a debounce mechanism.

- **digitalRead()** —

Reads the current state (HIGH or LOW) of a digital pin. I used it to detect when the button was pressed.

- **pinMode()** —

Configures a pin as either input or output. I used it to set up the pins connected to the button.

Auto Reset on Inactivity Implementation

1. Recorded the current time using millis() whenever the user interacted with the button.
2. Continuously compared the current millis() value with the last recorded activity time.
3. If the difference exceeded a set timeout (30 seconds), the program automatically reset the counter

4. What I Did

Hardware:

Connected a push button between pin 8 and GND.

Software:

Initialized serial communication, configured pin modes, and protected from debouncing with using millis(). The counter increases on each valid button press and the value is displayed on the Serial Monitor. Added an automatic reset at 20, a manual reset via the “reset” command, and an inactivity reset after 30 seconds using millis(). All functions were tested and verified successfully.

5. Lab work Evidence — Screenshots

```
// CSE107 - Prelab Homework #2
// Student ID: 240104004006

const int buttonPin = 8;
int counter = 0;
int buttonState = HIGH;
int lastButtonState = HIGH;
unsigned long lastDebounceTime = 0;
```

```

unsigned long debounceDelay = 50;
unsigned long lastPressTime = 0;

void setup() {
  Serial.begin(9600);
  pinMode(buttonPin, INPUT_PULLUP);
  Serial.println("Counter Started:");
}

void loop() {
  int reading = digitalRead(buttonPin);

  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }

  if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading == LOW && buttonState == HIGH) {
      counter++;
      lastPressTime = millis();
      Serial.print("Counter: ");
      Serial.println(counter);
    }
    buttonState = reading;
  }

  lastButtonState = reading;

  if (counter == 20){
    Serial.println("Counter Reached 20");
    counter = 0;
  }

  if (Serial.available() > 0) {
    String input = Serial.readStringUntil('\n');
    input.trim();
    if (input.equalsIgnoreCase("reset")) {

```

```
Serial.println("Reset command used:");  
counter = 0;  
}  
}  
  
if (millis() - lastPressTime >= 30000 && counter != 0) {  
    Serial.println("Inactive for 30s > Resetting");  
    counter = 0;  
    lastPressTime = millis();  
}  
}
```

Counter Started:

Counter: 1

Counter: 2

Counter: 3

Counter: 4

Counter: 5

Counter: 6

Counter: 7

Counter: 8

Counter: 9

Counter: 10

Counter: 11

Counter: 12

Counter: 13

Counter: 14

Counter: 15

Counter: 16

Counter: 17

Counter: 18

Counter: 19

Counter: 20

Counter Reached 20

Counter: 1

Counter: 2

Counter: 3

Reset command used:

Counter: 1

Counter: 2

Counter: 3

Counter: 4

Counter: 5

Inactive for 30s > Resetting

Counter: 1

Hardware:

